

למידת חיזוקים

בניגוד ל-MDP רגיל שבו אנחנו יודעים את כל ההתפלגויות, כאן אנחנו לא יודעים את ההתפלגויות של תוצאות הפעולות ושל המצבים.

הגדרה: סוכן פועל בסביבה במשך זמן:

- בזמן t הסוכן נמצא במצב s_t ובוחר פעולה a_t
- הסביבה מחזירה תגמול r_{t+1} ומצב s_{t+1} .
- האינטרציה ממודלת לפי MDP - (S, A, P, R, γ) - אבל ב-RL הסוכן לא יודע את P ו/או את R

המטרה: ללמוד פוליסה $\pi(a|s)$ שממקסמת את

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- בתרגול:
- נעבוד עם tabular RL setting - כלומר מספר המצבים והפעולות סופי ואפשר לשמור טבלה מלאה של זוגות state-action בזיכרון - כלומר $|S| \cdot |A|$ לא יותר מדי גדול. במציאות זה לא יהיה ככה.
 - נעבוד עם אופק אינסופי.

- הגדרות:
- למידה פאסיבית - צריך רק לגלות את הפוליסה שתמקסם את התועלת.
 - למידה אקטיבית - צריך למקסם את התועלת - מה שדורש לשלב בין למידת הפוליסה וניצול הפוליסה.
 - model-free - הסוכן לא משתמש בפונקציית המעברים P או פונקציית התגמולים R .
 - באופן כללי ב-RL לסוכן אין גישה ל- P ול- R - אבל ב-model-free הוא גם לא מנסה לשערך אותן, אלא הולך ישר לפוליסה.
 - ב-model-based הסוכן מנסה ללמוד את המודל ומתוך זה מחשב את הפוליסה.

- אלגוריתמים:
- ADP - אלגוריתם model-based
 - TD - אלגוריתם model-free

אלגוריתם Adaptive Dynamic Programming - ADP

נרצה לדגום כמה שיותר מקומות, ולכן לנסות דברים שונים. יהיו לנו 4 שלבים:

1. להריץ פוליסה בשביל לצבור נסיון
2. לשערך את פונקציית המעברים
3. לשערך את פונקציית התגמול
4. לשערך את הפוליסה

כשמריצים את הפוליסה, נתחיל עם באפר $\mathcal{B}$ בגודל $\mathcal{D}$, ונשתמש בפוליסה π במשך $\mathcal{D}$ צעדים
כאשר בכל צעד i :

$$\mathcal{B}[i] = (s_i, a_i = \pi(s_i), r_{i+1}, s_{i+1})$$

ואז השערוך של פונקציית המעברים הוא:

$$\hat{P}(s'|s, \pi(s)) = \frac{\text{\#times we saw } s \rightarrow s'}{\text{\#times we left } s}$$

ושערוך פונקציית התגמול הוא:

$$\hat{R}(s) = \text{average reward we got when reaching state } s$$

את שערוך הפוליסה מעדכנים בכל צעד:

$$V^\pi(s) \leftarrow \hat{R}(s) + \gamma \sum_{s'} \hat{P}(s'|s, \pi(s)) V^\pi(s')$$

נשים לב: V^π הוא הערך אם נרוץ אינסוף פעמים - ולכן γ גורם לו להתכנס:

$$G = R(1 + \gamma + \gamma^2 + \dots) = \frac{R}{1 - \gamma}$$

אלגוריתם TD - Temporal-Difference

ללמוד את המודל עם ADP זה מאוד יקר - כי יש המון מצבים וצריך להכפיל אותם בפעולות. אלגוריתם TD הוא model-free ולכן דורש פחות. בכל מעבר $s \rightarrow s'$ נעדכן:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha(R(s) + \gamma V^\pi(s') - V^\pi(s))$$

הערה: נוסחת online mean update

$$X_{n+1} = X_n + \frac{1}{n+1}(x_{n+1} - X_n)$$

מאפשרת לעדכן את הממוצע בלי לחשב אותו כל פעם מחדש.

זה דומה לנוסחת העדכון ל- V^π . אפשר לשים $\alpha = \frac{1}{n+1}$, אפשר להשתמש בערך קבוע, או אפשר לקבוע α לפי מספר הפעמים שביקרנו ב- s . כמות הstep שצריך לעשות הרבה יותר גדולה מאשר ADP - אבל החישוב של כל step הרבה יותר זול. במקום ערכי V נרצה להשתמש בערכי Q :

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi(s)) \cdot V^\pi(s')$$

$$Q^\pi(s, a) = R(s) + \gamma \sum_{s'} P(s'|s, a) \cdot V^\pi(s')$$

אם נהיה greedy אז ברגע שיש לנו Q^π , אפשר פשוט לבחור את a הכי טוב עבור כל s :

$$a^*(s) = \arg \max_a Q(s, a)$$

נסמן: ε exploration level. כמה נבצע פעולות באקראי, בשביל ללמוד.

discount factor γ

learning rate α

אם נשתמש ב-GLIE, באופן אינסופי, בהסתברות 1, נבחר את a ה-greedy. ערכי ה- Q יתכנסו לערכים האופטימליים.

למידה אקטיבית ופאסיבית

בלמידה פאסיבית עקבנו אחרי פוליסה קבועה, בניסיון לשערך את ה- V^π שלה. בלמידה אקטיבית צריך לאזן בין exploit ל-explore. יש לנו:

• target policy - שאנחנו רוצים למצוא כדי למקסם את תוחלת התגמול.

• behavior policy - שאנחנו משתמשים בה כדי ללמוד.

ככל שאנחנו לומדים, אנחנו בוחרים בbehavior policy, לומדים דרכה ערכים, ומעדכנים איתה את הtarget policy.

כשהtarget policy והbehavior policy הן אותו דבר, האלגוריתם נקרא on-policy: $\pi_b = \pi_{\text{target}}$. כש $\pi_b \neq \pi_{\text{target}}$ האלגוריתם הוא off-policy¹.

הגדרות: • episode הוא אינטרקציה שלמה אחת עם הסביבה.

Model-Based Active TD

• מתחילים עם מודל התחלתי

• בחורים פעולה באמצעות greedy או באמצעות GLIE על שערך ערכי Q :

$$\hat{Q}(s, a) = \hat{R}(s, a) + \gamma \sum_{s'} \hat{P}(s'|s, a) V(s')$$

• מבצעים את הפעולה

• מעדכנים את המודל - כלומר את:

$N(s, a, s')$ מספר המעברים עד כה (מגדילים ב1)

$\hat{R}$ שערך התגמולים

$\hat{P}$ פונקציית המעברים

אלגוריתם SARSA - On-Policy TD Control

• כאן אנחנו לא מאתחלים מודל - אלא מאתחלים את ערכי Q

- האלגוריתם הזה אינו model-based - לא לומדים את P ו R .

• בוחרים פעולה לפי exploration/exploitation.

• מבצעים את הפעולה.

• בוחרים את הפעולה הבאה. זה נותן לנו את:

$$(s, a, r, s', a')$$

• מבצעים את עדכון הSARSA:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

כלומר - הTD target הוא $r + \gamma Q(s', a')$, ומזיזים את $Q(s, a)$ לכיוון הtarget הזה.

¹זוהי תכונה של האלגוריתם - יש אלגוריתמים שהם on-policy ואלגוריתמים שהם off-policy.

• עוברים לזוג state-action הבא וחוזרים.

- נשים לב: האלגוריתם הזה הוא on-policy, בגלל שמשתמשים בערכי Q כדי לבחור את הפעולה.
- עלות העדכון היא $O(1)$.
- האלגוריתם הוא model-free.

Q-Learning

אנחנו רוצים לעשות exploitation, אבל עדיין ללמוד את הפוליסה האופטימלית. Q-Learning מבדיל בין איסוף המידע לבין השימוש בו לביצוע ה-exploitation. לכן במקום לשאול, כמו ב-SARSA, מה הפעולה הבאה שנבצע - אנחנו במקום זה שואלים מה הפעולה הכי טובה. כלומר מעדכנים לפי:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

בעצם, אנחנו תמיד מניחים שהאלגוריתם יעשה exploitation. העלות של העדכון היא $O(|A|)$.